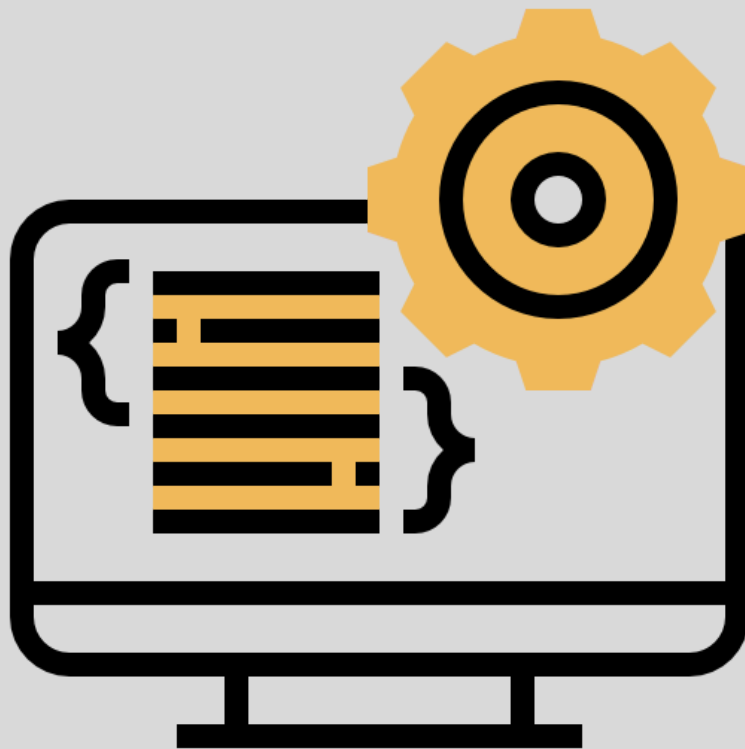


Trabajo Programación Multimedia y Dispositivos Móviles

Samuel Leyton Rojas
2 DAM



Tecnologías móviles

1) Explica la evolución de la red de comunicación móvil y las principales diferencias entre cada generación.

Las redes móviles se dividen en **generaciones (G)**:

- **1G (años 80)**: Solo voz analógica, calidad baja, sin seguridad.
- **2G (años 90)**: Llamadas digitales y SMS, más seguridad.
- **3G (años 2000)**: Navegación en internet, videollamadas, mayor velocidad.
- **4G (2010)**: Alta velocidad, streaming, apps pesadas, videollamadas HD.
- **5G (actual)**: Ultra rápida, baja latencia (ideal para IoT, coches autónomos, realidad virtual).

PRINCIPALES DIFERENCIAS -> funcionalidades nuevas, velocidad, calidad de audio

2) Android Vs iOS

a) Explica el modelo de capas de ambos Sistemas Operativos

- **ANDROID**
 - Kernel de linux : Es la capa que controla el hardware, la memoria, seguridad, procesos y drivers
 - Capa de Abstracción de Hardware (HAL) : Es el traductor entre el hardware y el sistema (Cámara, GPS, Sensores)
 - Android Runtime y librerías: Aquí se ejecutan las apps.

ART -> Corre las aplicaciones.

Librerías Nativas -> Permite a los programadores usar C/C++ para manejar cosas como bases de datos o gráficos.

NDK -> Permite a los programadores usar C/C++ cuando quieren apps más rápidas

Application Framework (Marco de aplicaciones) -> Es el kit de herramientas que los desarrolladores usan para crear apps (botones, pantallas, notificaciones)

- Marco de aplicaciones : Proporciona APIs para construir aplicaciones
- Aplicaciones del sistema y aplicaciones de usuario
Aplicaciones del sistema-> aplicaciones ya preinstaladas en el sistema como navegador web gestión de contactos.

Aplicaciones de usuario -> son aplicaciones descargadas por el usuario.

- **IOS** conexión del hardware
- Core services : Da servicios importantes para el funcionamiento de las aplicaciones
- Kernel de darwin : Es el corazón del sistema operativo y la capa más cercana al hardware. Incluye la gestión de la memoria, los procesos y el control del hardware.
- Core os : son los servicios de bajo nivel , se encarga de la seguridad redes y como servicios de red, bluetooth y almacenamiento.
- Cocoa touch : -> Es la capa superior y la más visible para el desarrollador y el usuario. Proporciona el marco de interfaz de usuario, los gestos multitáctiles, y el acceso a las aplicaciones y los servicios multimedia.

b) Indica el nombre del paquete de instalación de cada Sistema Operativo y el nombre de la tienda oficial en el que pueden descargarse este tipo de aplicaciones.

- Android

- Paquete: .apk (Android Package) → actualmente también se usa .aab (Android App Bundle).
-
- Tienda oficial: Google Play Store.
-
- iOS
-
- Paquete: .ipa (iOS App Store Package).
-
- Tienda oficial: App Store (de Apple).

c) ¿Qué lenguajes de programación se usan para el desarrollo de aplicaciones para cada SO?

Android

Java → el lenguaje clásico y más usado por años.

Kotlin → el recomendado oficialmente por Google (más moderno y seguro).

C/C++ → para partes de código con alto rendimiento (usando NDK).

Otros (multiplataforma): Dart (Flutter), JavaScript/TypeScript (React Native, Ionic).

iOS

Objective-C → el lenguaje histórico de Apple.

Swift → el lenguaje moderno y oficial recomendado por Apple.

C/C++ → para librerías o funciones de bajo nivel.

Otros (multiplataforma): Dart (Flutter), JavaScript/TypeScript (React Native).

3) En el caso de Android, indica qué es ART y qué es Dalvik.

Dalvik

Fue la máquina virtual original de Android.

Ejecutaba las aplicaciones traduciendo el código Java a un formato que Android pudiera correr.

Usaba el sistema JIT (Just In Time), que compilaba el código en el momento de la ejecución.

Funcionaba bien, pero consumía más batería y era más lento que su sucesor.

ART (Android Runtime)

Es la máquina virtual que reemplazó a Dalvik desde Android 5.0 (Lollipop).

Utiliza el sistema AOT (Ahead Of Time): compila las apps cuando se instalan, no en cada ejecución.

Ventajas:

Apps se ejecutan más rápido.

Mejor rendimiento y menor consumo de batería.

Depuración más sencilla para desarrolladores.

4) Explica las diferencias a la hora de desarrollar aplicaciones entre los siguientes acrónimos: UX, UI e IxD

1. UX (User Experience = Experiencia de Usuario)

Se centra en cómo se siente el usuario al usar la aplicación.

Tiene que ver con la usabilidad, la facilidad, la satisfacción y que el usuario logre lo que necesita sin frustraciones.

Ejemplo: Que en una app de comida sea fácil buscar, elegir y pagar un pedido.

2. UI (User Interface = Interfaz de Usuario)

Es el aspecto visual de la aplicación.

Incluye colores, tipografías, botones, menús, íconos y la disposición en pantalla.

Ejemplo: El diseño atractivo del botón "Comprar" o el estilo de la pantalla principal.

3. IxD (Interaction Design = Diseño de Interacción)

Se enfoca en cómo interactúa el usuario con la aplicación.

Define los flujos, los gestos, las animaciones y las respuestas del sistema.

Ejemplo: Qué pasa cuando deslizas hacia abajo (recargar contenido) o cuando tocas un ícono (se ilumina, vibra o abre una nueva pantalla).

5) Realiza una comparativa, indicando ventajas e inconvenientes de los siguientes frameworks. a) Xamarin b) Flutter c) React Native d) Ionic e) Nativescript

a) Xamarin

Ventajas:

Usa C# y .NET, ideal para quienes ya trabajan con tecnologías Microsoft.

Permite acceder directamente a las APIs nativas de iOS y Android.

Buen rendimiento porque compila a código nativo.

Inconvenientes:

El tamaño de las apps suele ser mayor.

Menos comunidad comparado con Flutter o React Native.

La parte visual puede requerir trabajo extra para que se vea igual en todas las plataformas.

b) Flutter

Ventajas:

Usa Dart y permite crear interfaces muy personalizadas con widgets.

Alto rendimiento, porque no depende de componentes nativos (usa su propio motor gráfico).

Una sola base de código para iOS, Android, web y escritorio.

Gran apoyo de Google y comunidad en crecimiento.

Inconvenientes:

El lenguaje Dart no es tan popular.

Las apps pueden pesar más que las nativas.

Integrar librerías nativas puede ser más complicado.

c) React Native

Ventajas:

Usa JavaScript/TypeScript, lenguajes muy populares.

Gran comunidad y soporte de Facebook (Meta).

Permite crear apps con rendimiento cercano al nativo.

Gran cantidad de librerías y plugins.

Inconvenientes:

Algunas funciones requieren escribir código nativo (Java, Swift, Objective-C).

El rendimiento puede bajar en apps muy complejas.

Puede haber problemas de compatibilidad tras actualizaciones.

d) Ionic

Ventajas:

Usa HTML, CSS y JavaScript, tecnologías web muy conocidas.

Fácil de aprender para desarrolladores web.

Permite publicar en web, iOS y Android con un solo código.

Gran ecosistema de componentes UI listos.

Inconvenientes:

No es completamente nativo, depende de WebView (apps menos fluidas).

Peor rendimiento en juegos o apps muy gráficas.

Experiencia de usuario menos "nativa".

e) NativeScript

Ventajas:

Usa JavaScript o TypeScript, con soporte también para Angular y Vue.

Acceso directo a las APIs nativas sin necesidad de plugins externos.

Verdadero código nativo, con mejor rendimiento que Ionic.

Inconvenientes:

Comunidad más pequeña que Flutter o React Native.

Documentación menos completa.

Menos popularidad = menos librerías y soporte.

6) ¿Qué tienen en común los frameworks anteriormente mencionados?

Multiplataforma, Lenguajes de alto nivel, Gran apoyo de comunidades y empresas

7) Enumera, quitando a iOS y Android, al menos 3 sistemas operativos móviles, indicando el año en el que debutaron.

Windows Phone → debutó en 2010

BlackBerry OS → debutó en 1999

Symbian OS → debutó en 1997

8) Investiga la proporción de SO móviles según su uso en la actualidad.

Cuota de mercado de sistemas operativos móviles (agosto de 2025)

Android: 73.9%

iOS: 25.71%

Samsung OS: 0.21%

KaiOS: 0.02%

Linux: 0.01%

Otros/Desconocido: 0.14%

9) Explica qué es Kotlin y para qué se utiliza

Es un lenguaje de programación de alto nivel, su principal uso es crear aplicaciones Android y es compatible con java solo que con una sintaxis más limpia y evitando algunos posibles errores.

10) Indica la importancia del uso de IDEs a la hora de desarrollar aplicaciones móviles.

1) Ahorro de tiempo

Combina varias herramientas (editor de código, compilador, depurador) en un solo lugar.

Permite probar y corregir errores rápidamente, sin cambiar de programa.

2) Organización del proyecto

Mantiene los archivos de la app, librerías y recursos bien estructurados.

Facilita el manejo de proyectos grandes o complejos.

3) Detección y corrección de errores

Incluye herramientas de depuración (debugging) que señalan errores en el código.

Algunas IDEs sugieren soluciones automáticamente.

4)Compatibilidad con dispositivos y sistemas

Permite emular dispositivos móviles (Android, iOS) para probar la app sin usar un teléfono real.

Facilita el compilado y empaquetado para diferentes sistemas operativos.

5)Integración con librerías y frameworks

Acceso fácil a SDKs, librerías externas y APIs, acelerando el desarrollo.

6)Mejor aprendizaje y productividad

Proporciona autocompletado, resaltado de sintaxis y documentación, lo que ayuda a programadores principiantes y avanzados.